

## Interrupts:

### Dealing with the Real World (ISRs)

Now, we must look at how **Hardware talks to Tasks** via **Interrupt Service Routines (ISRs)**.

When a physical event happens—like a button press, a timer alarm tripping, or a radio packet landing—the ESP32 hardware temporarily freezes the FreeRTOS scheduler, jumps directly to a special fast function called an **Interrupt Handler**, and then jumps back.

### ### The Golden Rule of Interrupts in FreeRTOS

Because an interrupt handler is running outside the normal scheduler task context, **you cannot call standard FreeRTOS APIs inside an ISR**.

Calling a function like `xQueueSend()` or `xSemaphoreGive()` from an interrupt will crash the CPU instantly.

Instead, FreeRTOS provides special, atomic versions of its functions specifically for interrupts, which always end in **``FromISR``**: `* xSemaphoreGiveFromISR() *`xQueueSendFromISR() *`vTaskNotifyGiveFromISR()`

## Code:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define JUMPER_GPIO_PIN 4

// Handle for the task that will process our heavy hardware events

TaskHandle_t hardware_processor_handle = NULL;

// 1 . The ISR - Runs in RAM
static void IRAM_ATTR gpio_isr_handler(void *arg){
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    // This directly unblocks our processing task at the hardware level.
    vTaskNotifyGiveFromISR(hardware_processor_handle,
    &xHigherPriorityTaskWoken);
    // If waking the target task triggers a context switch, force the CPU to
    switch immediately
    if(xHigherPriorityTaskWoken == pdTRUE){
        portYIELD_FROM_ISR();
    }
}
```

```

// 2 . The worker task (Consumer of the Interrupt)
void hardware_processor_task(void *pvParameters){
    printf("[Processor] Worker thread active. Entering blocked state...\n");
    while(1){
        // Block indefinitely until the hardware ISR sends a direct token
        signal
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        // This runs instantly when the interrupt triggers!
        printf("[Processor]  HARDWARE INTERRUPT DETECTED! Processing event
safely out of ISR context.\n");
    }
}

// 3 . Hardware Configuration
void init_interrupt_gpio(void){
    // Structure configuration for input pin
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << JUMPER_GPIO_PIN),
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE,           // Keep line high naturally
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_NEGEDGE             // Trigger when falling (High
to Low / Pressing Button)
    };
    gpio_config(&io_conf);
    // Installing the ESP32 global ISR routing service
    gpio_install_isr_service(0);
    // Linking our specific handler function up to GPIO 4
    gpio_isr_handler_add(JUMPER_GPIO_PIN, gpio_isr_handler, NULL);
}

void app_main(void){
    printf("[SYSTEM] Booting High-Speed ISR Architecture...\n");
    // spawning the worker thread first to capture its ref handle address
    xTaskCreate(
        hardware_processor_task,
        "Processor Task",
        2048,
        NULL,
        10, // High priority to preempt background tasks instantly
        &hardware_processor_handle
    );
    // Initialize the physical interrupt lines
    init_interrupt_gpio();
    while(1){
        printf("Other processes...\n");
        vTaskDelay(1500/ portTICK_PERIOD_MS);
    }
}

```

```
}
```

## Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

W (241) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size
I (255) main_task: Started on CPU0
I (255) main_task: Calling app_main()
[SYSTEM] Booting High-Speed ISR Architecture...
[Processor] Worker thread active. Entering blocked state...
Other processes...
Other processes...
Other processes...
Other processes...
Other processes...
Other processes...
Other processes...
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
Other processes...
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
Other processes...
Other processes...
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
[Processor]  HARDWARE INTERRUPT DETECTED! Processing event safely out of ISR context.
```

- ❑ **The Background Noise:** The `app_main` loop is happily spinning inside a `while(1)`, printing `Other processes....`
- ❑ **The Hardware Attack:** The absolute microsecond your wire hits GND, the ESP32 hardware forces a hard stop. It fires the code marked with `IRAM_ATTR` at lightning speed.
- ❑ **The Preemption Cut-In:** Because your `hardware_processor_task` has a priority of **10** (while your main loop is much lower), the kernel doesn't wait for `app_main` to finish its line or hit its delay. It freezes it on the spot and immediately swaps the CPU context to print `[Processor] ⚡ HARDWARE INTERRUPT DETECTED!`.
- ❑ **The Safe Return:** The moment the processor task finishes printing, it blocks itself again, waiting for the next notification. The kernel then seamlessly jumps back to the main task exactly where it left it.